

Ausarbeitung zum Thema: OOP

Strukturierung

1. Das objektorientierte Paradigma
 - a. Definition
 - b. Historische Entwicklung
 - c. Ziele
2. Zentrale Konzepte
 - a. Klassen und Objekte
 - b. Kapselung
 - c. Vererbung
 - d. Polymorphie
 - e. Abstraktion
3. Szenarien und Interaktionen
 - a. Szenarien
 - b. Interaktionen
4. Bedeutung von OOP in Studium und Praxis
 - a. Verbreitung
 - b. Relevanz in der Softwareentwicklung
5. Herausforderungen beim Erlernen von OOP
 - a. Typische Fehlvorstellungen
 - b. Lernschwierigkeiten
 - c. Erkenntnisse aus der Programmierdidaktik

Definition

Quelle: <https://masters.donntu.ru/2009/fvti/gordeev/library/article6.pdf>

- Konzentration auf die verhaltensbezogenen und strukturellen Merkmale von Entitäten als vollständige Einheiten
- Konzeptzentriert, indem es sich auf alle Arten von Merkmalen fokussiert, die ein bestimmtes Konzept ausmachen
- Paradigma unterstützt und umfasst die 4 Säulen (Grundprinzipien):
 - Abstraktion (Abstraction):
 - Bildung von Modellen oder Repräsentationen, indem Gemeinsamkeiten und Unterschiede einer Menge von Objekten betrachtet werden
 - Ziel: herausfiltert der wesentlichen und für alle Elemente relevanten Eigenschaften, während unwichtige oder nur unterscheidende Merkmale ausgeblendet werden
 - Entstehung einer allgemeinen Darstellung, die die charakteristischen Eigenschaften aller Objekte zusammenfasst
 - Kapselung (Encapsulation):
 - Bündelung von Daten und Funktionen sowie das Verbergen von Implementierungsdetails
 - Unterstützung der Abstraktion, durch klare Differenzierung zwischen der Objektbeschreibung (was ein Objekt ist und was es tut) und seiner konkreten Umsetzung (wie es realisiert wird)

- Vererbung (Inheritance):
 - Möglichkeit, bestehende Klassen oder Repräsentationen wiederzuverwenden und darauf aufbauend neue Klassen zu definieren
 - Übernahme gemeinsamer Eigenschaften und Funktionen
 - Reduktion von Redundanzen
 - Erhöhung der Wiederverwendbarkeit des Codes
- Polymorphie (Polymorphism):
 - Fähigkeit, neue Klassen als Varianten bestehender Klassen zu definieren
 - Unterschiedliche Implementierungen können bereitgestellt werden, während die gleiche Schnittstelle / Spezifikation erhalten bleibt
 - Einheitliche Methode kann somit je nach Objekt unterschiedliche Verhaltensweisen ausführen

Historie

Quelle: <https://www.sciencedirect.com/science/article/pii/S0890540113000795>

- Grundlage für die objektorientierte Programmierung stellte die Sprache Simula (ursprünglich eine Sprache zur Prozessbeschreibung und Simulation)
- Entwickelt von Dahl und Nygaard
- Erster Entwurf von Simula 1963, heute rückblickend als Simula I bezeichnet
- Kerngedanke war, dass nicht der Quellcode selbst die Realität modelliert, sondern die Objekte, welche durch den Code erzeugt, werden
- Fertigstellung des ersten Compilers 1965
- Fertigstellung der Sprache im Jahr 1967 unter dem Namen Simula 67
- Einführung zentraler Konzepte, die später die Grundlage der objektorientierten Programmierung bildeten:
 - Klassen
 - Objekte
 - Vererbung
 - Dynamische Objektinstanzen
- Aus Ideen Entwicklung modernes objektorientiertes Paradigma (später Beeinflussung von Programmiersprachen wie Smalltalk, C++ und Java)
- Simula = Beschreibung der gemeinsamen Grundideen der beiden Sprachversionen
- Simula I & Simula 67 = konkrete Sprachversionen
- Simula 67 als die Programmiersprache, die die entscheidenden Grundlagen der objektorientierten Programmierung geschaffen hat
- Ermöglichung neuer Programmierkonzepte durch Simula 67:
 - Strukturierte Datenmodelle
 - Datenabstraktion
 - Prozesse
 - Vererbung
 - Modularisierung
- Objektorientierte Programmierung entsteht durch die neuen Denk- und Modellierungsmöglichkeiten, die diese Konzepte eröffneten
- Gleichzeitige Einführung von objektorientierten Frameworks (nicht mehr das Anwendungsprogramm steuert die Bibliothek, sondern das Framework übernimmt die Kontrolle über den Programmablauf)
- Prinzip namens „Inversion of Control“ (Umkehrung der Kontrolle)

- Framework entscheidet, wann die vom Entwickler bereitgestellten Komponenten ausgeführt werden
- Ende der 1980er Jahre gewinnt die objektorientierte Programmierung an Bedeutung in der Industrie
- Entstehung zahlreicher objektorientierter Sprachen, die unterschiedliche Schwerpunkte hinsichtlich Klassen, Objekten, Vererbung und weiterer Konzepte setzten
- 1981 veröffentlicht Alan Snyder einen einflussreichen Artikel „The Essence of Objects“ von Hewlett Packard
- Beschreibende und nicht vorschreibende Arbeit der Gemeinsamkeiten bestehender objektorientierter Systeme (im Gegensatz zu Simula und Smalltalk)
- Beschreibung und Klassifizierung der Merkmale zeitgenössischer objektorientierter Programmiersprachen (bspw. Smalltalk, C++, Common Lisp Object System und Xerox Star, usw.)
- Wesentliche Konzepte nach Snyder:
 - Objekt verkörpert eine Abstraktion
 - Objekte stellen Dienste bereit
 - Clients richten Anfragen an ihre diese Dienste
 - Objekte sind gekapselt
 - Anfragen identifizieren Operationen
 - Anfragen können Objekte identifizieren
 - Neue Objekte können erzeugt werden
 - Dieselbe Operation kann bei unterschiedlichen Objekten unterschiedliche Implementierungen und beobachtbar unterschiedliches Verhalten besitzen
 - Objekte können anhand ihrer Dienste klassifiziert werden (Schnittstellenhierarchie)
 - Objekte können Implementierungen gemeinsam nutzen
 - Objekte können eine gemeinsame Implementierung teilen (mehrere Instanzen)
 - Objekte können teilweise Implementierungen teilen (Implementierungsvererbung oder Delegation)

Ziele

Quelle: <https://dl.acm.org/doi/epdf/10.1145/382192.383004>

- Softwaretechnische Ziele von OOP:
 - Technologie von Softwarekomponenten:
 - Entstehung, wenn ein Berechnungsproblem in Teilprobleme zerlegt wird („Divide & Conquer“-Prinzip), die gemeinsam zur Lösung des Gesamtproblems beitragen
 - Dazu zählen: Funktionen, Prozeduren, Objekte sowie nebenläufig ausführbare Komponenten wie Prozesse, Akteure und Agenten
 - Funktionen, Prozeduren und Objekte unterscheiden sich hinsichtlich ihrer Funktionalität, sind jedoch Dienstanbieter, die über eine Schnittstelle verfügen, welche die für Klienten bereitgestellten Dienste beschreibt, sowie über einen Rumpf (*Body*), der diese Dienste implementiert
 - Objektorientierte Softwarebibliotheken:
 - Sammlungen von Softwarekomponenten, die als wiederverwendbare Bausteine für die Softwareentwicklung dienen

- Programmiersprachen dienen als Bindeglied, um Bibliothekskomponenten miteinander zu kombinieren
- Komponenten von OOP-Bibliotheken sind Klassen, aus denen Objekte erzeugt werden → grundlegende Unterscheidung der Eigenschaften der Komponenten, das Verständnis von Komposition & Art Verbindungscode
- Zwei Ebenen der Komposition:
 - Deklarative Komposition von Klassen
 - Komposition von Klassen zur Laufzeit
- OOP-Bibliotheken enthalten hierarchisch organisierte Sammlungen von Klassen, deren Beziehungen und gemeinsame Nutzung durch Vererbung bestimmt werden
- Wiederverwendbarkeit und kapitalintensive Softwaretechnologie:
 - Kapitalgüter sind wiederverwendbare Ressourcen
 - Kosten über ihre Nutzung amortisiert
 - Wiederverwendbarkeit umfasst die Bildung von physischem als auch von konzeptionellem Kapital (kann sowohl wirtschaftlich als auch intellektuell begründet werden)
 - In der Softwaretechnologie hängt die Kapitalbildung von Konzepten, Modellen und Softwareinfrastrukturen ab und nicht von physischen Produktionsanlagen
 - Um die Prozesse der Kapitalbildung in der Softwareentwicklung zu verstehen, müssen verschiedene Formen der Wiederverwendbarkeit unterschieden werden:
 - Anwendungsübergreifende Wiederverwendbarkeit
 - Entwicklungsbezogene Wiederverwendbarkeit
 - Programm-Wiederverwendbarkeit
 - Code-Wiederverwendbarkeit
- Objektorientierte Programmierung im sehr großen Maßstab (Megaprogrammierung):
 - „groß“ beschreibt die Ausdehnung in mehreren Dimensionen (siehe Figur 5)

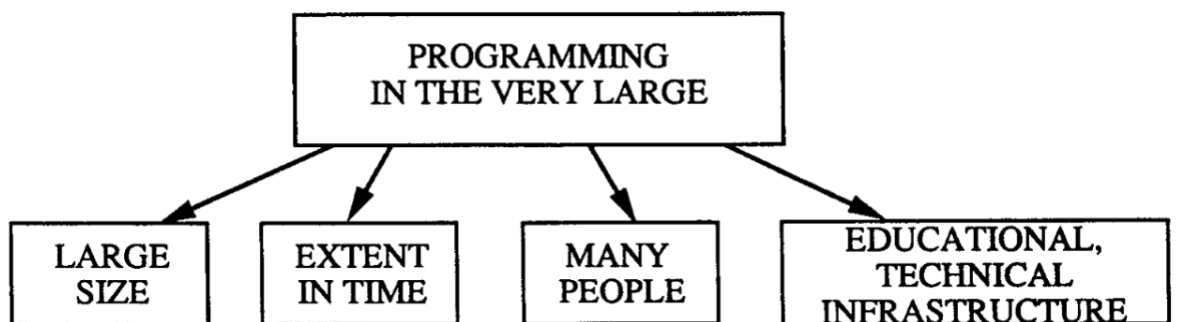


Figure 5: Dimensions of Largeness

- Erläuterung:
 - **Programmgröße:** Verwaltung umfangreicher Programme durch Modularisierung, Konsistenzprüfungen von Schnittstellen und weitere Softwarewerkzeuge zur Beherrschung ihrer physischen Größe
 - **Zeit:** große und gute Programme benötigen lange Entwicklungszeiten
 - **Menschen:** Managementherausforderungen bei der Zusammenarbeit

- **Bildungs- und technische Infrastruktur:** Bildungsinfrastruktur ist entscheidend für den Erfolg von Projekten im großen Maßstab
- Objektorientierte Programmierung bietet einen Rahmen für die nahtlose Verwaltung großer Hardware-, Software und Anwendungssysteme
- OOP im sehr großen Maßstab erweitert die traditionelle OOP wie folgt:
 - OOP im sehr großen Maßstab:
 - = OOP + Nebenläufigkeit + Persistenz
 - = OOP + Mehrrechnersysteme + mehrere beteiligte Personen

Quelle: <https://www.sciencedirect.com/science/article/pii/S0167642312001918>

- Verfolgung des Ziels, die hohe Komplexität von Softwaresystemen besser beherrschbar zu machen und den kognitiven Aufwand sowie die geistige Belastung von Entwicklern und Wartungspersonal zu reduzieren

Zentrale Konzepte

Quelle: <https://masters.donntu.ru/2009/fvti/gordeev/library/article6.pdf>

- Klassen und Objekte dienen dazu, reale oder abstrakte Entitäten innerhalb eines Softwaresystems zu modellieren
- Objekte:
 - Repräsentationen von Entitäten
 - Kapselung struktureller Eigenschaften als auch Verhaltensweisen einer Entität
 - Strukturelle Eigenschaften:
 - Beschreibung durch Attribute
 - Attribute speichern Informationen über Objekt & bestimmen dessen möglich Zustände
 - Bsp.: Objekt der Klasse „Student“ könnte die Attribute Name, Matrikelnummer und Semester besitzen
 - Jedes Objekt hat eindeutige Identität und stellt konkrete Instanz einer Klasse dar
 - Objekte sind somit abstrahierte Entitäten, die sowohl Zustand (Attribute) als auch Verhalten (Operationen) kapseln
- Klassen:
 - Beschreibung einer Menge von Objekten mit einer gemeinsamen Implementierung
 - Festlegung, welche Attribute und Methoden die daraus erzeugten Objekte besitzen und wie diese umgesetzt werden
 - Klasse definiert gemeinsame Struktur und gemeinsame Verhalten ihrer Objekte, enthalten sind:
 - Gemeinsame Attribute
 - Implementierungen von Methoden
 - Semantische Bedeutungen
 - Beziehungen zu anderen Klassen
 - Interaktionsmöglichkeiten mit anderen Objekten
 - Bsp.: Klasse Student definiert Attribute Name und Semester & die Methode einschreiben(). Studierende wie „Max Mustermann“ sind Objekte dieser Klasse.
- Typen:
 - Definition der gemeinsamen Spezifikation von Objekten
 - Typ legt fest:

- Welche Attribute müssen vorhanden sein
- Welche Methoden sind verfügbar
- Welche Bedeutung besitzen diese
- Wie Objekte miteinander interagieren können
- Konzentration auf die Frage, was ein Objekt können muss (Klassen beschreiben, wie dies technisch umgesetzt wird)
- Typen können auf unterschiedliche Weise mit Klassen verbunden sein:
 - Explizit → wenn Klasse vorgegebene Schnittstelle implementiert
 - Implizit → wenn Klasse selbst Schnittstelle def. & Umsetzung bereitstellt
- Links:
 - Konkrete Repräsentationen von Beziehungen zwischen einzelnen Objekten
 - Verbinden von Objekten
 - Darstellung der Instanzen einer Assoziation, abstrahierte Beziehungen zwischen Objekten
 - Bsp.: zwischen Objekt der Klasse Student und einem Objekt der Klasse Kurs ein Link bestehen, welcher ausdrückt, dass der Student an diesem Kurs teilnimmt.
- Assoziationen:
 - Beziehungen zwischen Klassen
 - Definition der gemeinsamen Struktur und Bedeutung von Links
 - Festlegung, wie Objekte unterschiedlicher Klassen miteinander verbunden sein können
 - Formen der Assoziationen:

Form	Beschreibung	Beispiel
Aggregation	- Ganzes-Teil-Beziehung zwischen Objekten (lose Besitzbeziehung) - Objekt besteht aus mehreren Bestandteilen, die jedoch auch unabhängig vom Ganzen existieren können	- Bibliothek enthält mehrere Bücher - Bücher können auch ohne die Bibliothek weiterhin existieren
Komposition	- Spezielle Form der Aggregation mit starker Besitzbeziehung - Lebensdauer der Teilobjekte ist direkt an das Gesamtobjekt gekoppelt	- Haus besteht aus Räumen - Wird das Haus gelöscht, existieren die Räume nicht mehr
Generalisierung (Vererbung)	- Taxonomische Beziehung zwischen allgemeineren und spezielleren Klassen - Neue Klasse aus bereits existierender - Spezialisierte Klasse übernimmt die Eigenschaften und Methoden der allgemeineren Klasse	Klasse Fahrzeug - Attribute: Geschwindigkeit - Methode: fahren() Klasse Auto - Erbt von Fahrzeug - Ergänzt die Eigenschaft AnzahlTüren

- Polymorphie:
 - Durch Vererbung wird Polymorphie ermöglicht
 - Dieselbe Methode bzw. Schnittstelle unterschiedliche Implementierungen besitzen

- Bsp.: Aufruf der Methode bewegen()
 - Auto bewegt sich durch Fahren
 - Flugzeug bewegt sich durch Fliegen
 - Schiff bewegt sich durch Fahren auf dem Wasser

Quelle: *Quelle: <https://www.sciencedirect.com/science/article/pii/S0890540113000795>*

- Idee von Wegner: Objektorientierung wird durch die Existenz von Klassen charakterisiert
- Klassen befinden sich relativ zu Objekten auf einer Metaebene, da eine Klasse die Implementierung und das Verhalten jener Objekte beschreibt, die wir Instanzen einer Klasse nennen
- Bereitstellung einer Fabrik (Prozedur oder Methode), die diese Beschreibung umsetzt, durch Erzeugung einer Instanz
- Einfacher Objekte direkt im Programm zu beschreiben, wenn man nur weniger benötigt
- Grundidee von Shelf, Emerald, NewtonScript und JavaScript: Betrachtung des Objekts und nicht die Klasse als grundlegendes Konzept
- Klassen dennoch nicht nutzlos → können in ihrer Rolle als Objekterzeuger selbst durch Objekte repräsentiert werden
-

Szenarien und Interaktionen

Quelle: *<https://masters.donntu.ru/2009/fvti/gordeev/library/article6.pdf>*

- Beschreibung von Abläufen und Kommunikationsvorgängen zwischen Objekten
- Modellierung, wie verschiedene Objekte innerhalb eines Systems miteinander zusammenarbeiten zur Aufgabenerfüllung
- Szenarien:
 - Konkrete Darstellungen von Ereignisabläufen zwischen Objekten
 - Beschreibung einer Folge von Nachrichten bzw. Methodenaufrufen, die zwischen mehreren Objekten ausgetauscht werden
 - Instanz einer Interaktion
 - Zeigt, wie ein bestimmter Prozess im System abläuft
 - Bsp.: Student meldet sich zu einem Kurs an
 1. Objekt Student sendet Anfrage an das Objekt Kurs
 2. Objekt Kurs prüft die verfügbaren Plätze
 3. Objekt Kurs bestätigt die Anmeldung
 4. Objekt Student erhält die Rückmeldung
- Interaktionen:
 - Allgemeine Muster von Szenarien mit einer gemeinsamen Struktur oder Implementierung
 - Definition, welche Objekte beteiligt und in welcher Richtung Nachrichten ausgetauscht werden
 - Festlegung, wie die Kommunikation zwischen Objekten grundsätzlich abläuft, während ein Szenario eine konkrete Ausführung des Ablaufs darstellt
- Nachrichten:
 - Kommunikation zwischen objektorientierten Systemen
 - Zwei Rollen:
 - Sender: Objekt, das eine Anfrage stellt

- Empfänger (Receiver): Objekt, das die Anfrage empfängt und verarbeitet
- Sender ruft beim Empfänger eine Methode auf, indem er eine Nachricht sendet
- Nachricht entspricht der Methodenschnittstelle (Operation Interface)
- Tatsächliche Ausführung innerhalb des Empfängers durch die konkrete Methodenimplementierung (Operation Implementation)
- Bsp.: student.anmelden(kurs);

Bedeutung von OOP

Quelle: <https://www.sciencedirect.com/science/article/pii/S0890540113000795>

- Einführung des objektorientierten Frameworks war wegweisend für die moderne Softwareentwicklung
- viele heutige Technologien basieren auf diesem Prinzip der Umkehrung der Kontrolle:
 - GUI-Frameworks
 - Web-Frameworks
 - Game Engines
 - Simulationssysteme
 - Mobile Anwendungsframeworks
- Entwickler müssen nicht mehr den gesamten Programmablauf selbst steuern
- Framework stellt Infrastruktur bereit, während lediglich die anwendungsspezifischen Objekte und deren Verhalten definiert werden

Herausforderungen

Quelle: <https://www.sciencedirect.com/science/article/pii/S0167642312001918>

- Verwechslung der Vererbungsrichtung
- Selbst professionelle Softwareentwickler haben Schwierigkeiten mit den grundlegendsten Konzepten und Prinzipien der Objektorientierung
- Schwierigkeiten können mittels der „Dual-Process-Theorie“ (= in unserem Denken wirken zwei getrennte kognitive Mechanismen, die grob dem entsprechen, was gemeinhin als intuitives und analytisches Denken bezeichnet wird) dargelegt werden
- Entstehung von Fehlern durch den Konflikt zwischen Denkmechanismen
- Teilweise Unterschätzung der Schwierigkeit der Erlernung der OOP
- Grundlegendes Merkmal der OOP, dass Entwickler dem Verständnis des Problems mehr Aufmerksamkeit widmen können, indem sie sich auf Objekte konzentrieren
- organisieren die Problemdarstellung anhand Objekte, die an Aktivitäten Problems beteiligt sind
- Erfordern einer sorgfältigen Analyse der semantischen Struktur des Problems
- Förderung der Konzentration auf das Problem, durch Unterstützung eines länger anhaltenden Prozesses des Problemverständnisses
- Herausforderungen:
 - Definition von Klassen auf Grundlage identifizierter Objekte
 - Objektorientierte Programmiersprachen definieren eine Klasse als Beschreibung einer Menge von Objekten
 - In der realen Welt existieren Dinge auch unabhängig von Klassen

- Anfänger verfolgen häufig einer Methode, die sie in Programmierkursen gelernt haben, wo sie zuerst Entitäten der Anwendungsdomäne und dann Klassen und Methoden identifizieren
- Anfänger und Experten legen oft das Klassenmodell fest, bevor die Methoden implementiert werden (Folge: Verwerfung der früh entworfenen Klassen)
- Anfänger nehmen oft an, dass Instanzen von Klassen automatisch erzeugt werden → Verwendung von Instanzen von Klassen, ohne sie zuvor tatsächlich erzeugt zu haben
- Zerlegung und Zuordnung von Funktionalität zu bestimmten Objekten
 - Schwierigkeiten für Anfänger in der Verbindung zwischen deklarativen und prozeduralen Aspekten einer Lösung
 - Schaffen es nicht große Prozeduren in kleinere funktionale Einheiten zu zerlegen, sondern ordnen die gesamte Prozedur einer Klasse zu
 - Anfänger verbringen viel Zeit damit, Domänentitäten zu erstellen und wieder zu verwerfen, bevor sie mit der Definition von Objekten beginnen
 - Mangelndes Wissen über OOP-Konzepte sowie die Übertragung ungeeigneter Denkweisen aus prozeduralen Programmiersprachen verhindern häufig die Erstellung eines korrekten komplexen Plans und die sinnvolle Verteilung der darin enthaltenen Aktionen auf Klassen
 - Folge: oft Überarbeitung der statischen Aspekte der Gesamtlösung
- Vererbung
 - Natürliche Hierarchien verfügen über eine grundlegende Abstraktionsebene, auf der Gemeinsamkeiten zwischen realen Objekten besonders zuverlässig und schnell erkannt werden
 - Keine Existenz einer solchen Unterscheidung im OOP
 - Unterschiede zwischen objektorientierter Vererbung und der Art und Weise, wie Menschen Vererbungsbeziehungen natürlicherweise interpretieren
 - Softwarekomponenten, an denen Vererbungsbeziehungen beteiligt sind, weisen dreimal mehr Fehler auf
 - Anfänger versuchen so häufig wie möglich das Prinzip der Vererbung anzuwenden und es dabei ungeeignet verwenden
 - Anfänger haben bereits Schwierigkeiten mit der Vererbung statischer Eigenschaften haben
 - Noch größere Probleme beim Umgang mit der Vererbung von Funktionalitäten bzw. Methoden
- Zielsystem
 - Entwickler, die mit dem System vertraut sind, organisieren die Zerlegung eines neuen Problems anhand ihres Wissens über bestehende Klassenstrukturen
 - Förderung das Ziel der Code-Wiederverwendung
 - Entwickler, die mit dem System nicht vertraut sind, können hingegen durch die vermeintliche Natürlichkeit der objektbasierten Zerlegung dazu verleitet werden, Entwürfe mit unnötigen und möglicherweise ineffizienten Unterscheidungen sowie fehlerhaften Vererbungsstrukturen zu erstellen
- Klassenkonzept
 - Anfänger betrachten eine Klasse oft als eine Menge von Objekten
 - Schreiben dem Klassenkonzept daher Eigenschaften einer Menge zu

Siehe Experteninterview weitere Herausforderungen.

Quellen

<https://masters.donntu.ru/2009/fvti/gordeev/library/article6.pdf>

<https://www.sciencedirect.com/science/article/pii/S0890540113000795>

<https://dl.acm.org/doi/epdf/10.1145/382192.383004>

<https://www.sciencedirect.com/science/article/pii/S0167642312001918>